Emotion Recognition from Faces Project Report

Executive Summary

This report consolidates the full end-to-end workflow of the project: model development, evaluation, optimization, interpretability, class balancing, adversarial robustness, and deployment.
The core application performs facial emotion recognition on FER2013 and exposes the final model through a Gradio dashboard with webcam and video audit support.

Project Objectives

- Train and compare deep learning models for seven-class facial emotion recognition.
- Optimize the chosen model for efficient deployment through ONNX and Core ML export, pruning, and quantization.
- Analyze attention behavior in a fine-tuned Vision Transformer.
- Study class imbalance handling through focal loss, ADASYN, and mixup.
- Evaluate adversarial robustness under FGSM and PGD, then improve it with adversarial training.
- Deploy the final system through a Gradio app prepared for Hugging Face Spaces.

Repository Scope

The implementation is organized across notebooks for experimentation, reusable task-specific modules in src/, trained checkpoints and result summaries in results/, and the deployment app in app/gradio_app.py.

Methodology and Experimental Flow

Data and Problem Setup

The project uses FER2013-style seven-class emotion classification: angry, disgust, fear, happy, sad, surprise, and neutral.
Experiments progress from data inspection and augmentation into model training, evaluation, targeted analysis tasks, and finally deployment.

Baseline Modeling Workflow

- Data loading and preprocessing are handled through reusable dataset utilities in src/core/dataset.py.
- Training logic is centralized in src/core/train.py so that MobileNetV3, ViT, and later task-specific variants share a consistent optimization pipeline.
- Evaluation notebooks and result summaries capture confusion matrices, accuracy, F1 metrics, and exported checkpoints.

Model Selection Outcome

MobileNetV3 emerged as the main deployment candidate because it offered the strongest trade-off between accuracy, model size, and deployment suitability.
The strongest cross-entropy MobileNetV3 checkpoint reached 67.96% validation accuracy with macro F1 0.6504, while the selected deployment checkpoint remained close at 67.43% validation accuracy with macro F1 0.6424.
A fine-tuned ViT was retained for interpretability analysis, where its attention structure could be inspected more directly than the CNN baseline.

Key sources: notebooks 01-05, results/summaries/*.md, src/core/*, src/models/*.

Task 1: Optimization and Deployment Readiness

What Was Done

Task 1 focused on converting the trained MobileNetV3 model into portable formats and benchmarking whether pruning and quantization preserved usable performance.

- Exported the model to ONNX and Core ML.
- Created an INT8 quantized Core ML variant.
- Benchmarked latency and size across PyTorch, ONNX, Core ML FP32, and Core ML INT8 artifacts.
- Added a notebook-integrated sanity check so Core ML and ONNX outputs can be compared on the same random input.

Quantitative Results

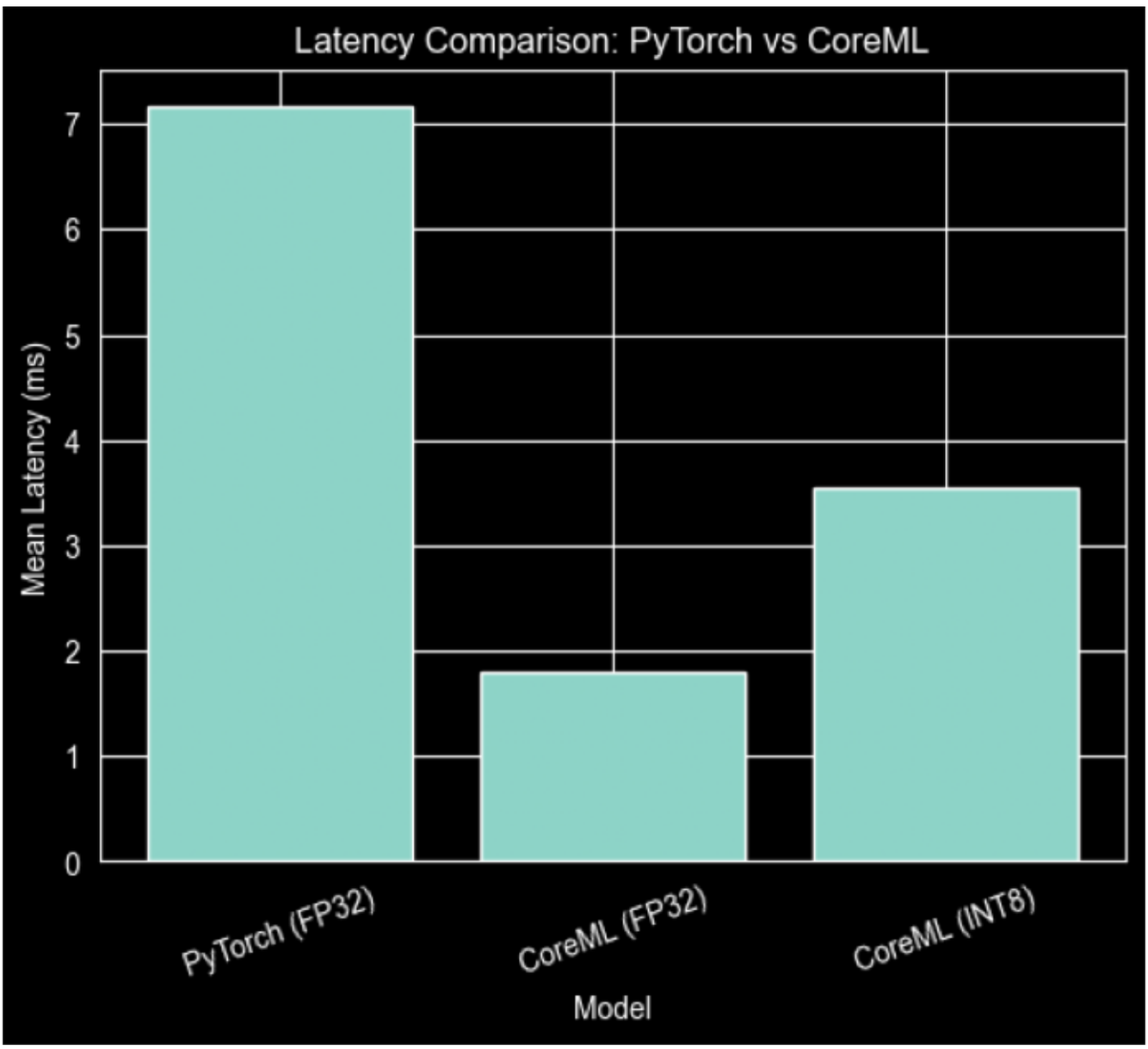
- PyTorch FP32: 7.16 ms mean latency, 67.26% accuracy, 6.92 MB.
- Core ML FP32: 1.79 ms mean latency, 67.40% accuracy, 6.79 MB.
- Core ML INT8: 3.55 ms mean latency, 66.75% accuracy, 1.78 MB.

Interpretation

Core ML FP32 delivered the best latency, about 3.99x faster than the PyTorch baseline, while maintaining essentially the same accuracy.
INT8 quantization reduced model size by about 3.8x with only a small accuracy drop, making it attractive for mobile deployment where storage matters.
The separate results.py script was kept as a numerical sanity check rather than a benchmark so it does not get confused with the formal latency evaluation.

Files: src/task_1_optimization/*, notebooks/08_coreml_analysis.ipynb, results/coreml_benchmark.md.

Task 1 Visual: Core ML Benchmark



This benchmark figure summarizes the deployment motivation for the project.

- Core ML FP32 is the lowest-latency deployment target in the study.
- Quantized Core ML slightly sacrifices accuracy but provides a much smaller package.
- These results justified packaging the app for local and Hugging Face deployment around the MobileNetV3-based pipeline.

Source: notebooks/08_coreml_analysis.ipynb output.

Task 2: Attention Analysis with ViT

What Was Done

Task 2 examined the final transformer block of a fine-tuned ViT-small model to understand which attention heads were most informative for emotion recognition.

- Extracted last-block multi-head attention maps from the ViT checkpoint.
- Computed per-emotion mean attention maps.
- Measured mutual information head scores globally and per emotion.
- Produced emotion-specific summaries of the highest-MI heads.
- Added ablation analysis to compare the effect of masking low-MI and high-MI heads.

Results

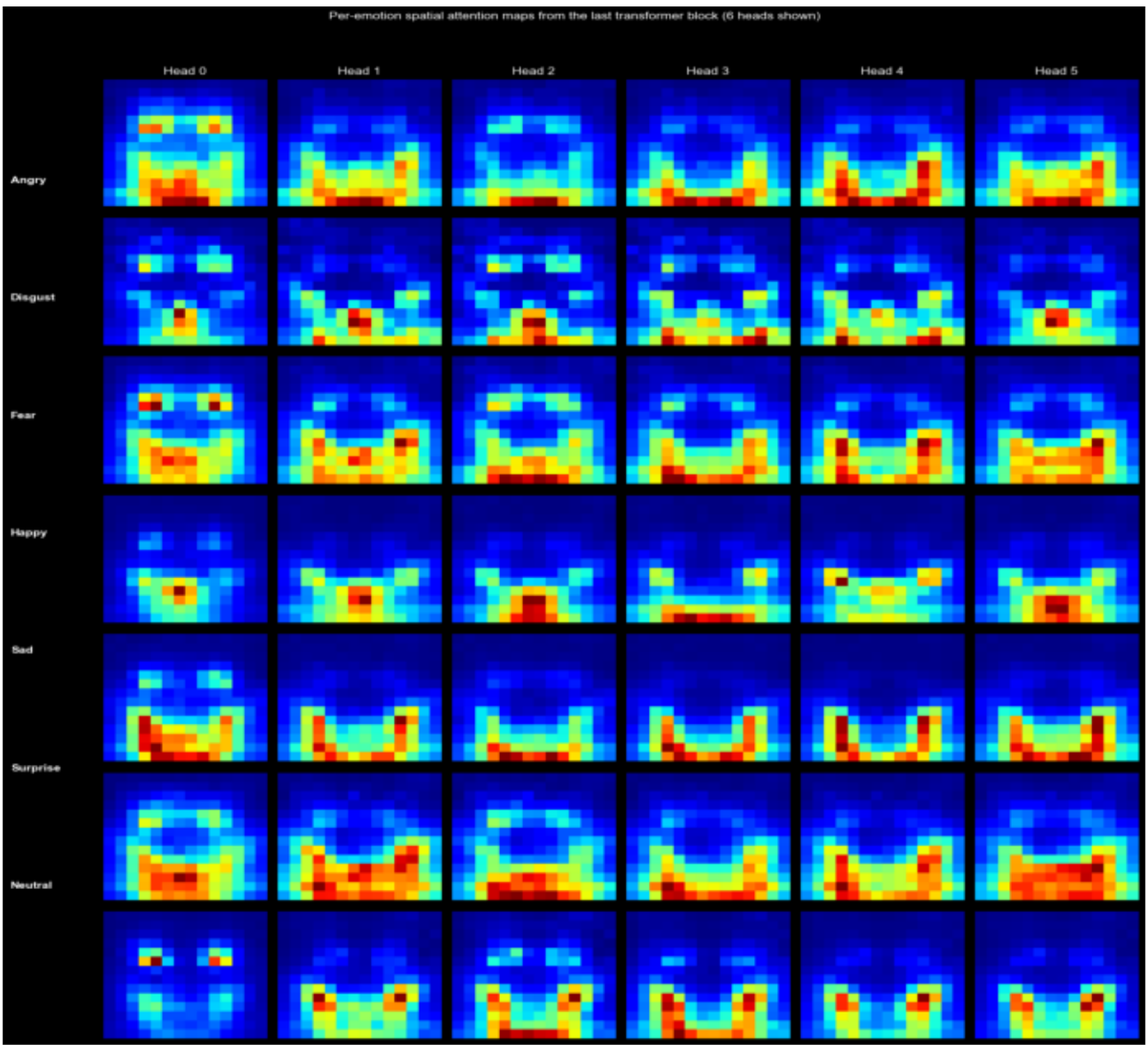
- Baseline accuracy in the notebook analysis: 0.6452.
- Masking low-MI heads: 0.6452.
- Masking high-MI heads: 0.6452.
- Top MI heads: [3, 4, 2].

Interpretation

The analysis pipeline now satisfies the required interpretability steps, including per-emotion heatmaps and class-aware summaries.
The current qualitative pattern shows attention concentrating mostly around mouth and lower-face regions, with smaller secondary focus around nose and cheeks.
The ablation result is structurally valid but weak in effect size, so the notebook is best treated as a first-pass interpretability study rather than strong causal proof of head importance.

Files: notebooks/07_attention_analysis.ipynb, src/task_2_attention/*, results/attention_summary.md.

Task 2 Visual: Emotion-Conditioned Attention Maps



Each row corresponds to an emotion and each column corresponds to a head from the last transformer block.

- The figure makes it possible to compare whether distinct emotions drive different spatial attention patterns.
- The notebook was updated so the row-to-emotion mapping is clearly labeled, matching the task instructions.
- In practice, attention remains fairly concentrated in lower-face regions, which is plausible for expression-sensitive cues such as smiles, frowns, and lip tension.

Source: notebooks/07_attention_analysis.ipynb output.

Task 3: Class Balancing Strategies

What Was Done

Task 3 evaluated three common approaches for class imbalance: focal loss, ADASYN oversampling, and mixup augmentation.
Each strategy was trained and then compared using macro F1, micro F1, and minority-class F1 to ensure the final conclusion was not dominated by frequent classes alone.

Results

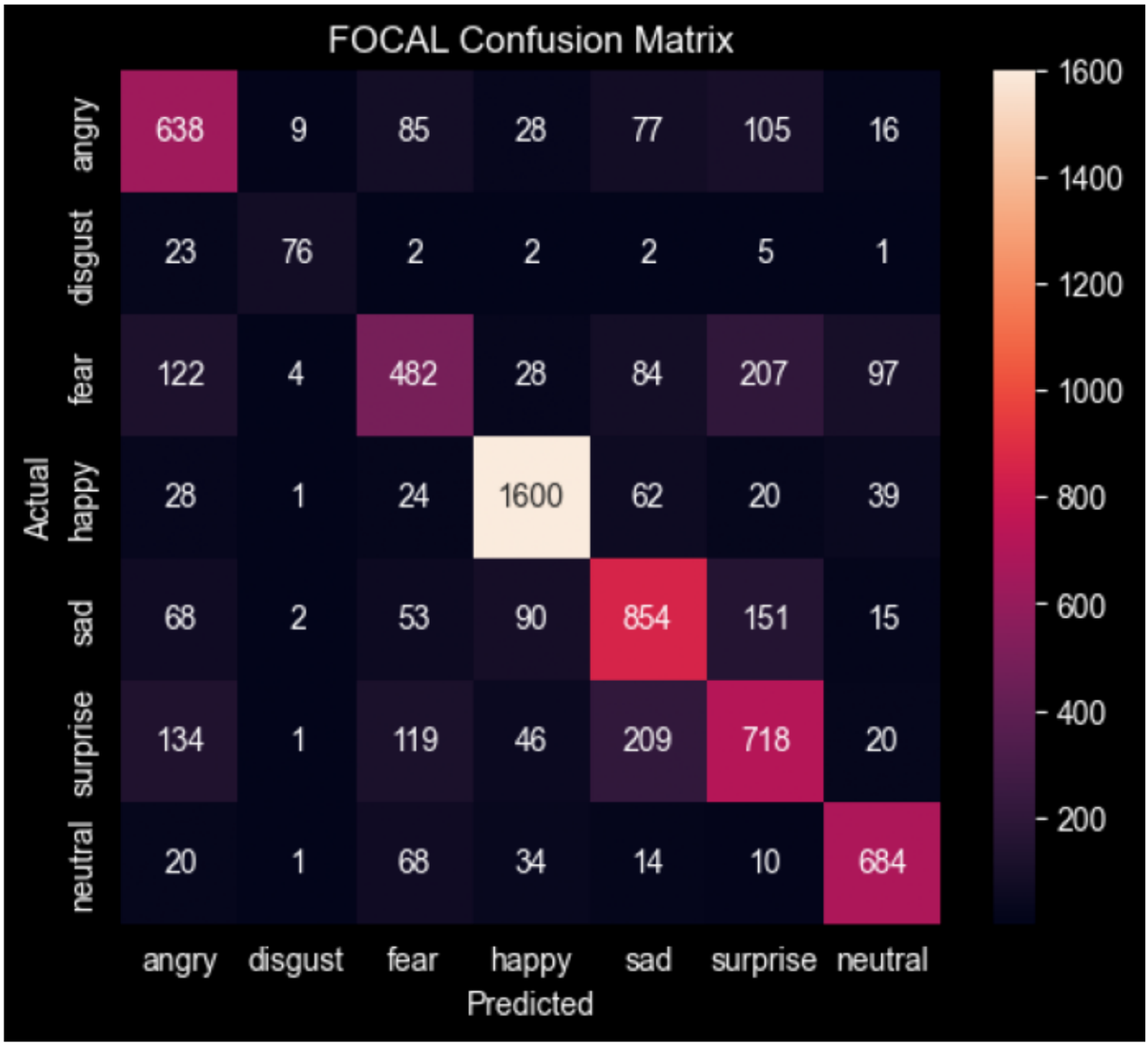
- Focal loss: macro F1 0.6928, micro F1 0.7038, minority F1 0.6303.
- Mixup: macro F1 0.6891, micro F1 0.7108, minority F1 0.5987.
- ADASYN: macro F1 0.6285, micro F1 0.6592, minority F1 0.5196.

Interpretation

Focal loss produced the most balanced outcome overall and the strongest minority-class performance in this project.
Mixup remained competitive and slightly stronger on micro F1, but it underperformed focal loss on the more important macro and minority metrics.
ADASYN helped class balancing conceptually but was clearly weaker than the other two strategies on this dataset and model setup.

Files: notebooks/06_class_balancing.ipynb, src/task_3_balancing/*, results/balancing_comparison.md.

Task 3 Visual: Confusion Matrix for a Balanced Training Variant



Representative confusion-matrix output from the class-balancing experiments.

- These matrices were used alongside aggregate F1 scores to see which emotions benefited from rebalancing.
- The main value of this task was not just raising a single metric, but improving stability on minority expressions that are typically harder to classify.

Source: notebooks/06_class_balancing.ipynb output.

Task 4: Adversarial Robustness

What Was Done

Task 4 studied how fragile a face emotion classifier becomes under small input perturbations, then measured whether limited adversarial training improved robustness.

- Rebuilt the notebook around FER2013 dataloaders instead of synthetic placeholders.
- Configured FGSM with epsilon = 8/255.
- Configured PGD with epsilon = 8/255, alpha = 1/255, and 10 steps.
- Updated the attack implementation to operate in unnormalized pixel space and then renormalize before inference, so the epsilon values remain defensible.
- Injected PGD-based adversarial training into roughly 10% of training batches.

Results

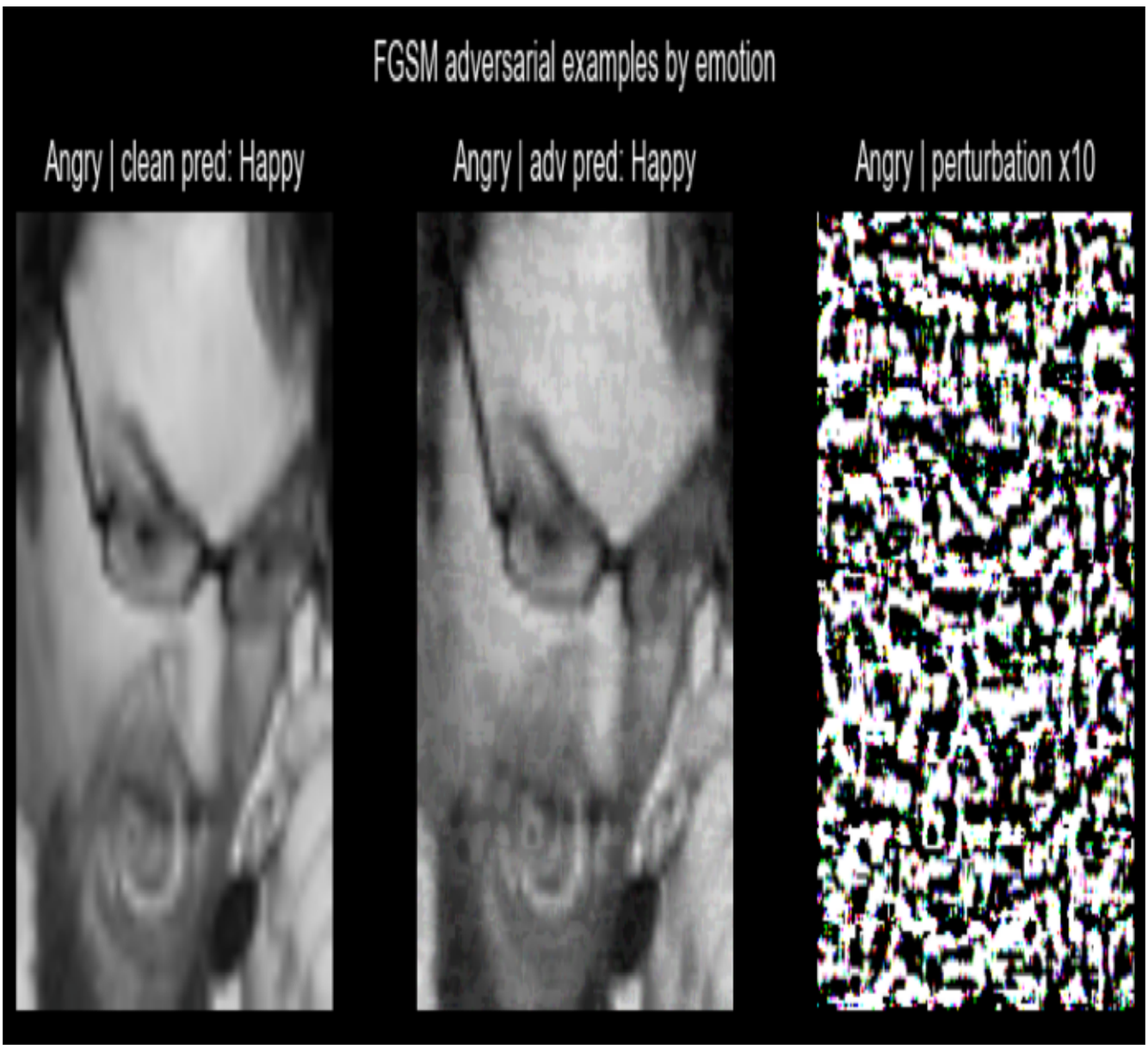
- Clean accuracy: 0.5339 before, 0.5995 after adversarial training.
- FGSM accuracy: 0.0132 before, 0.0857 after.
- PGD accuracy: 0.0000 before, 0.0061 after.

Interpretation

The baseline model was highly vulnerable to perturbations, especially under PGD, which effectively collapsed accuracy.
Adversarial training improved robustness consistently, although the model remained far from robust in a strong worst-case sense.
This task demonstrates that robustness is measurable and partially improvable, but it remains a challenging open problem relative to clean accuracy optimization.

Files: notebooks/09_adversarial_analysis.ipynb, src/task_4_adversarial/*, results/adversarial_results.md.

Task 4 Visual: FGSM Adversarial Examples



This notebook visualization pairs clean and adversarial samples to show how small perturbations alter model behavior.

- The perturbations are visually subtle, which is why formal attack metrics are necessary rather than visual inspection alone.
- The notebook was updated so attack generation is numerically consistent with normalized dataloaders while preserving standard FGSM and PGD settings.
- The examples support the quantitative conclusion that even modest perturbations can trigger wrong emotion predictions.

Source: notebooks/09_adversarial_analysis.ipynb output.

Task 5: Dashboard, Fairness Monitoring, and Deployment

What Was Done

The final task transformed the trained model into a usable application through a Gradio dashboard and a deployment configuration suitable for Hugging Face Spaces.

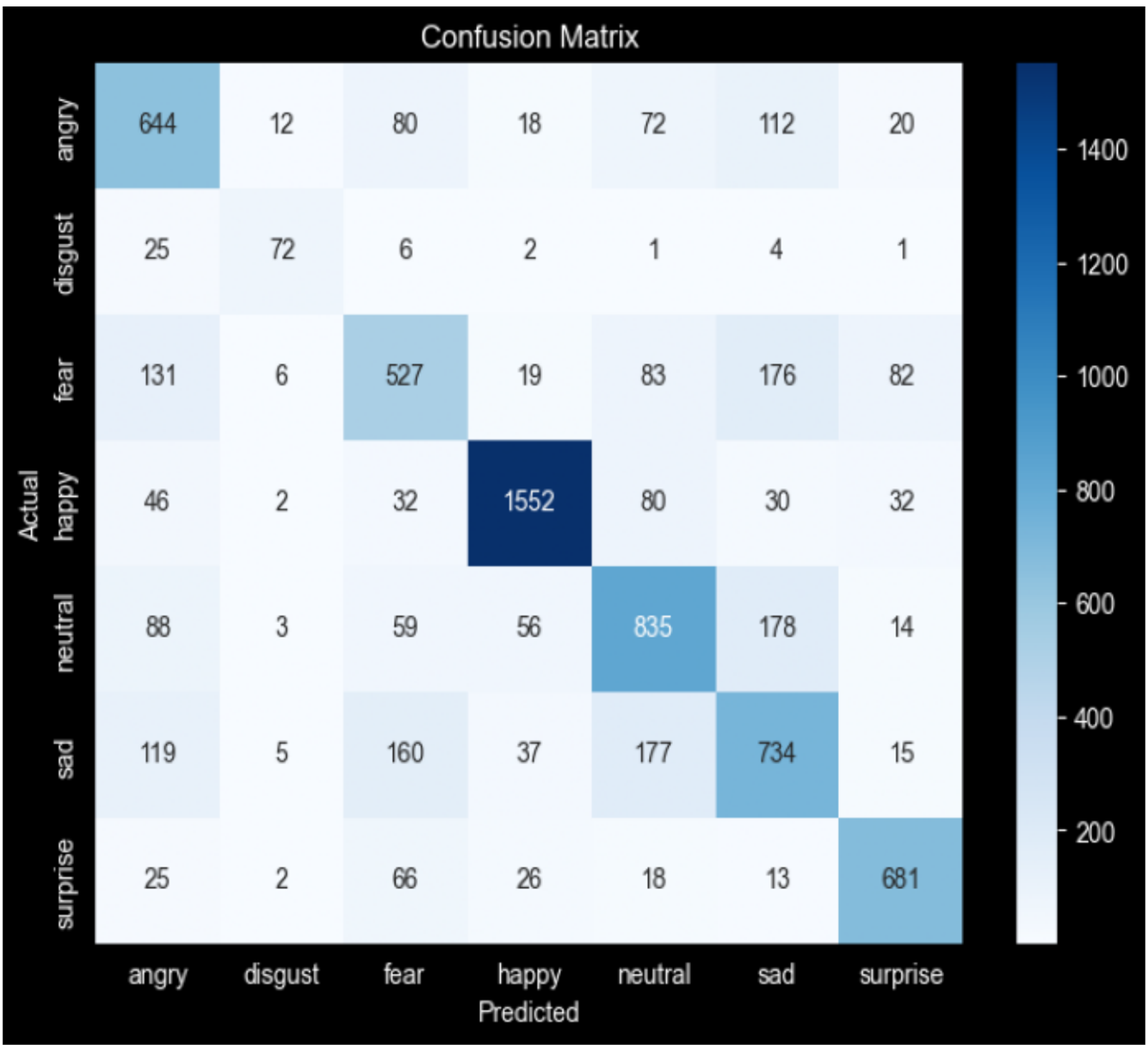
- Fixed Gradio compatibility issues so the app runs reliably on local localhost without changing the core inference logic.
- Corrected the MobileNetV3 checkpoint loading so the app uses the same architecture as training.
- Added webcam monitoring, video audit, logging, and sliding-window fairness metrics.
- Added a Hugging Face Spaces entrypoint and packaging files for deployment.
- Modified the Space path so checkpoints can be downloaded from a public Hugging Face model repository instead of embedding binary weights directly in the Space repository.

Practical Outcome

The app now supports local use and a deployment-friendly structure with app.py, packages.txt, and a Gradio-first entrypoint.
The dashboard exposes emotion prediction, visualization, audit outputs, and fairness-oriented summaries in one place.
This task connects the research-style notebook work to a real interface, which is important for demonstrating the project beyond offline experimentation.

Files: app/gradio_app.py, app.py, packages.txt, README.md, Hugging Face Space/model repos.

Model Evaluation Visual: Confusion Matrix



A standard evaluation view from the core training workflow is included here because it anchors the later task-specific analyses.

- It shows the practical pattern of which emotions are more easily confused in the baseline recognition problem.
- The later tasks build on this baseline by improving efficiency, interpretability, balance, robustness, and deployment readiness rather than replacing the core classifier objective.

Source: notebooks/05_evaluation.ipynb output.

Cross-Task Conclusions

Main Findings

- MobileNetV3 was the most deployment-ready model, with strong accuracy and excellent portability to ONNX and Core ML.
- Core ML optimization delivered substantial latency gains with minimal accuracy loss, validating the mobile deployment path.
- Focal loss was the best class-balancing strategy in this project, especially for minority-class handling.
- The ViT attention workflow satisfied the interpretability requirements and produced plausible region-focused observations, even though head ablation effects were weak.
- The adversarial analysis showed severe vulnerability under FGSM and PGD, while adversarial training provided measurable but limited gains.
- The Gradio dashboard successfully integrated inference, logging, auditing, and deployment preparation into a usable application layer.

Limitations

- Attention ablation did not show strong separation between low-MI and high-MI heads.
- Robustness remained low against strong iterative attacks even after adversarial training.
- Fairness metrics in the dashboard depend on reference labels or audit assumptions, so they should be interpreted as monitoring aids rather than definitive fairness guarantees.

Recommended Next Steps

- Train the robust model longer or with stronger adversarial curricula.
- Improve attention-region attribution with finer facial landmark alignment.
- Evaluate the dashboard with a non-placeholder demographic pipeline if fairness auditing is a formal requirement.
- Package the report findings with saved notebook outputs for final submission reproducibility.

This report synthesizes the implemented code, saved result summaries, and notebook visual outputs in the repository.